

Lifecycle Model

[Previous: Overview](#) | [Documentation Home](#) | [Next: Entity Definitions](#)

Coredeux uses one small lifecycle vocabulary so services, modules, and future features all speak the same language.

Canonical Operations

The framework currently uses these operations:

- CREATE
- MODIFY
- UPSERT
- DELETE
- FETCH

They are exposed through `EntityLifecycleContext` and are intended to stay stable across the framework.

Why Lifecycle Exists

The framework should not make callers manually load old values, build internal context objects, or decide which modules should run.

Instead, Coredeux derives the lifecycle state internally and passes it to validators, hooks, audit handlers, and any future modules that need the same information.

Operation Semantics

Save

`save(entity)` currently has two lifecycle moments:

- before persistence: CREATE
- after persistence: UPSERT

When possible, Coredeux also loads the current persisted state so old and new values can both be present in context.

Update

`update(entity)` uses MODIFY.

This is a strict update path. The framework expects the entity to already exist and resolves the persisted state before applying the change.

Remove

`remove(entity)` and `remove(id, type)` use DELETE.

The framework resolves the stored entity before delete so modules can react to the persisted state.

Fetch

`load`, `loadAll`, `query`, and `refresh` use FETCH.

These are read-oriented framework flows.

Phase Versus Operation

Operation is the business meaning.

Phase is the point in the framework flow where the work happens.

Examples:

- before-save
- after-save
- before-update
- after-update
- before-delete
- load
- before-refresh
- after-refresh

That distinction matters because modules often care about when they run, not just what the high-level operation is.

Runtime Context

Coredeux derives the execution context internally.

RequestContext

`RequestContext` holds request-scoped metadata when a servlet request is available. It is typically derived from headers, parameters, or attributes.

Common values include:

- request id
- correlation id
- user id
- tenant or site id
- locale

OperationContext

OperationContext is the framework-owned runtime container passed to modules.

It typically includes:

- invokedAt
- requestContext
- lifecycleContext

EntityLifecycleContext

EntityLifecycleContext contains the transition data:

- operation
- identifier
- oldValue
- newValue

Old And New Values

Coredeux derives old and new values itself.

General rules:

- create-like flow
 - oldValue may be null
 - newValue is the incoming entity
- modify flow
 - oldValue is the stored entity
 - newValue is the incoming entity
- delete flow
 - oldValue is the stored entity
 - newValue is typically null
- fetch flow
 - entity state is made available through the current execution context

Module Execution And Lifecycle

Modules use the lifecycle context to keep behavior consistent.

Examples:

- validators inspect the incoming entity and lifecycle intent
- hooks react to framework phases

- audit handlers read operation, identifier, old value, and new value without application code building an audit payload manually

Stability Guidance

New features should reuse the existing lifecycle vocabulary instead of inventing a separate operation model.

That keeps the framework predictable for both human developers and future agent-driven tooling.

[Previous: Overview](#) | [Documentation Home](#) | [Next: Entity Definitions](#)